



ImageNet Dataset

ImageNet is a large-scale, hierarchical image dataset developed by researchers at Stanford and Princeton (led by Fei-Fei Li) to advance research in **visual recognition** and **machine learning**.

- **Launched:** 2009
 - **Purpose:** To provide a comprehensive labeled image resource for training and evaluating models in **image classification**, **object detection**, and **image segmentation**.
-



Key Features

1. Large-Scale Dataset

- Contains **14+ million images**, each manually labeled with object categories.
- Over **22,000 categories** (called **synsets**) based on the **WordNet hierarchy**.
- Example: "Dog" synset contains sub-synsets like **Labrador**, **Bulldog**, etc.

2. Diverse Categories

- Covers a wide range: animals, objects, scenes, plants, tools, vehicles, etc.

3. High-Quality Annotations

- All labels are **human-annotated** for reliability and accuracy.

4. Hierarchical Structure

- Based on the **WordNet hierarchy**.
- Each node = **synset** (a set of synonymous words representing a concept).

5. ILSVRC (ImageNet Large Scale Visual Recognition Challenge)

- Annual competition (2010–2017) based on a subset of ImageNet.
 - Contains:
 - **1,000 categories**
 - **1.2M training images**
 - **50k validation**
 - **100k test images**
 - Catalyzed breakthroughs in image classification and object detection.
-

Impact on Deep Learning

1. Catalyst for CNN Revolution

- **AlexNet (2012)** won ILSVRC with a deep CNN, reducing top-5 error from 25% to **15%**.
- Sparked the deep learning boom in computer vision.

2. Birth of State-of-the-Art Architectures

Benchmarked and developed using ImageNet:

Model Name	Year
LeNet-5	1998
AlexNet	2012
VGG-16	2014
GoogLeNet (Inception v1)	2014
Inception v3	2015
ResNet-50	2015
Inception-ResNet	2016
Inception v4	2016
DenseNet	2017
MobileNet	2017
Xception	2017
NASNet	2018
EfficientNet	2019
Vision Transformer (ViT)	2020
Swin Transformer	2021
ConvNeXt	2022

3. Transfer Learning

- Pretrained ImageNet models are widely used as **feature extractors**.
- Fine-tuned on smaller datasets (e.g. **medical, satellite, custom datasets**).
- Enables efficient learning where **labeled data is scarce**.

Applications of ImageNet

- **Image Classification:** Predict the correct label for an image.
- **Object Detection:** Detect and locate objects with bounding boxes.

- **Image Segmentation:** Assign a class to each pixel.
- **Research & Benchmarking:** Standard benchmark for evaluating CV algorithms.

Advantages

- **Scale and Diversity:** Covers a wide array of object types with millions of samples.
- **High Annotation Quality:** Human-labeled, ensuring better learning performance.
- **Benchmark Standard:** Became the **de facto** standard in CV benchmarking.

Limitations

Limitation	Description
Dataset Bias	Skewed toward certain categories or regions. May affect generalization.
Domain Specificity	Lacks enough data for specialized domains (e.g., medical imaging).
Legal/Ethical Issues	Concerns about copyright, licensing, and privacy in commercial use.

Summary

ImageNet transformed the landscape of computer vision by:

- Enabling **deep CNN architectures**.
- Serving as a universal **benchmark**.
- Supporting the growth of **transfer learning**. It remains a foundational tool in developing high-performing vision models.

LeNet-5 – Pioneering Convolutional Neural Network

What is LeNet-5?

LeNet-5 is one of the earliest and most influential convolutional neural network (CNN) architectures, developed by **Yann LeCun et al. in 1998**. It was designed to recognize **handwritten digits** from the MNIST dataset. This model laid the groundwork for modern deep learning in computer vision.

Features of LeNet-5

- Introduced the idea of combining **convolution + pooling + fully connected layers**.

- Uses **tanh activation functions** (ReLU wasn't common then).
 - Inspired architectures like **AlexNet, VGGNet, ResNet**.
 - Low computational cost, designed for devices with limited power in the 90s.
 - Introduced **sparse connectivity** between layers to reduce complexity.
-



Architecture of LeNet-5

1. Input Layer

- **Input Size:** $32 \times 32 \times 1$ (Grayscale)
 - MNIST images (28×28) were zero-padded to 32×32 .
-

2. Convolutional Layer (C1)

- **Filters:** 6 filters of size 5×5
 - **Output:** $28 \times 28 \times 6$
 - **Stride:** 1
 - **Activation:** tanh
-

3. Subsampling Layer (S2) – Average Pooling

- **Filter:** 2×2
 - **Stride:** 2
 - **Output:** $14 \times 14 \times 6$
 - **Activation:** tanh
-

4. Convolutional Layer (C3)

- **Filters:** 16 filters of size 5×5
 - **Output:** $10 \times 10 \times 16$
 - **Stride:** 1
 - **Activation:** tanh
 - **Connectivity:** Sparse – not every input map is connected to every output map
-

5. Subsampling Layer (S4) – Average Pooling

- **Filter:** 2×2
 - **Stride:** 2
 - **Output:** $5 \times 5 \times 16$
 - **Activation:** tanh
-

6. Fully Connected Layer (C5)

- **Input:** Flattened $5 \times 5 \times 16 = 400$
- **Output:** 120
- **Activation:** `tanh`

7. Fully Connected Layer (F6)

- **Input:** 120
- **Output:** 84
- **Activation:** `tanh`

8. Output Layer

- **Input:** 84
- **Output:** 10 classes (digits 0–9)
- **Activation:** `softmax`



Summary of LeNet-5 Architecture

Layer Type	Feature Maps	Output Size	Kernel Size	Stride	Activation
Input	1	32×32	-	-	-
Conv (C1)	6	28×28	5×5	1	tanh
AvgPool (S2)	6	14×14	2×2	2	tanh
Conv (C3)	16	10×10	5×5	1	tanh
AvgPool (S4)	16	5×5	2×2	2	tanh
FC (C5)	120	1×1	-	-	tanh
FC (F6)	84	1×1	-	-	tanh
Output	10	1×1	-	-	softmax



Development History & Timeline

Year	Milestone
1988	First neural network used on ZIP code digits
1989	First LeNet versions for digit recognition
1990	LeNet using backpropagation on simple networks
1994	LeNet-4 introduced sparse connections
1998	LeNet-5 officially published (most cited)
~2005	Replaced in production by deeper CNNs like AlexNet

Applications

- Digit recognition (e.g., ZIP codes, bank checks)
- Early OCR systems
- Academic learning and baseline model for CNNs
- Embedded systems with low compute power

Influence & Subsequent Work

- Inspired deep architectures like **AlexNet, VGG, GoogLeNet, ResNet**
- Basis for CNN design patterns: Conv → Pool → FC
- Introduced **sparse connections** and **parameter sharing**
- Proof that CNNs could outperform traditional ML for vision

AlexNet – The Breakthrough CNN Model (2012)

What is AlexNet?

AlexNet is a deep convolutional neural network developed by **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton**, which won the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** in **2012** by a **huge margin**. It was the first model to prove the power of deep learning on large-scale image classification, marking a turning point in computer vision.

Features of AlexNet

- Consists of **8 layers**: 5 convolutional + 3 fully connected.
- First to use **ReLU activation** instead of tanh or sigmoid (improves training speed).
- Applied **dropout** to reduce overfitting.
- Used **data augmentation** and **GPU acceleration**.
- **Overlapped max pooling** instead of traditional non-overlapping pooling.
- Trained on **ImageNet dataset** with 1.2 million images across 1000 classes.

Architecture of AlexNet

1. Input Layer

- **Input Size:** $227 \times 227 \times 3$ (RGB image)
 - MNIST images were resized and normalized to fit this dimension.
-

2. Convolutional Layer (Conv1)

- **Filters:** 96
 - **Filter Size:** 11×11
 - **Stride:** 4
 - **Padding:** 0
 - **Output Size:** $55 \times 55 \times 96$
 - **Activation:** ReLU
-

3. Max Pooling Layer (Pool1)

- **Filter Size:** 3×3
 - **Stride:** 2
 - **Output Size:** $27 \times 27 \times 96$
-

4. Convolutional Layer (Conv2)

- **Filters:** 256
 - **Filter Size:** 5×5
 - **Stride:** 1
 - **Padding:** 2 (Same padding)
 - **Output Size:** $27 \times 27 \times 256$
 - **Activation:** ReLU
-

5. Max Pooling Layer (Pool2)

- **Filter Size:** 3×3
 - **Stride:** 2
 - **Output Size:** $13 \times 13 \times 256$
-

6. Convolutional Layer (Conv3)

- **Filters:** 384
 - **Filter Size:** 3×3
 - **Stride:** 1
 - **Padding:** 1
 - **Output Size:** $13 \times 13 \times 384$
 - **Activation:** ReLU
-

7. Convolutional Layer (Conv4)

- **Filters:** 384
 - **Filter Size:** 3×3
 - **Stride:** 1
 - **Padding:** 1
 - **Output Size:** $13 \times 13 \times 384$
 - **Activation:** ReLU
-

8. Convolutional Layer (Conv5)

- **Filters:** 256
 - **Filter Size:** 3×3
 - **Stride:** 1
 - **Padding:** 1
 - **Output Size:** $13 \times 13 \times 256$
 - **Activation:** ReLU
-

9. Max Pooling Layer (Pool3)

- **Filter Size:** 3×3
 - **Stride:** 2
 - **Output Size:** $6 \times 6 \times 256$
-

10. Fully Connected Layer (FC6)

- **Input Size:** $6 \times 6 \times 256 = 9216$ (flattened)
 - **Output Size:** 4096
 - **Activation:** ReLU
-

11. Fully Connected Layer (FC7)

- **Input Size:** 4096
 - **Output Size:** 4096
 - **Activation:** ReLU
-

12. Fully Connected Layer (FC8) – Output Layer

- **Input Size:** 4096
 - **Output Size:** 1000 (ImageNet classes)
 - **Activation:** Softmax
-



Summary of AlexNet Architecture

Layer Type	Feature Maps	Output Size	Kernel Size	Stride	Padding	Activation
Input	3	227×227	-	-	-	-
Conv1	96	55×55	11×11	4	0	ReLU
Pool1	96	27×27	3×3	2	-	-
Conv2	256	27×27	5×5	1	2	ReLU
Pool2	256	13×13	3×3	2	-	-
Conv3	384	13×13	3×3	1	1	ReLU
Conv4	384	13×13	3×3	1	1	ReLU
Conv5	256	13×13	3×3	1	1	ReLU
Pool3	256	6×6	3×3	2	-	-
FC6	4096	1×1	-	-	-	ReLU
FC7	4096	1×1	-	-	-	ReLU
FC8 (Output)	1000	1×1	-	-	-	Softmax

Development History & Timeline

Year	Milestone
2010	ILSVRC (ImageNet Challenge) begins
2012	AlexNet wins ILSVRC with top-5 error of 15.3%
2012	Used 2 GPUs for training large-scale model
2012+	Deep learning becomes dominant in vision tasks

Applications

- Large-scale image classification
- Object detection (used in R-CNN)
- Feature extractor for transfer learning
- Face and gesture recognition
- Medical imaging, robotics, surveillance

Influence & Subsequent Work

- Inspired models like **VGGNet, GoogLeNet, ResNet**
- Introduced **ReLU, Dropout, Overlapping Pooling**
- Sparked the deep learning revolution in CV
- Paved the way for deeper and wider CNNs

Key Innovations in AlexNet

1. ReLU Activation Function

- ReLU (Rectified Linear Unit) introduced non-linearity without saturating gradients.
- ReLU accelerates convergence during training compared to traditional activation functions like **sigmoid** or **tanh**.
- Helped the network train faster and more efficiently.

2. Dropout Regularization

- Applied to fully connected layers.
- Randomly sets a fraction of neuron outputs to zero during training.
- Prevents overfitting by discouraging complex co-adaptations on training data.

3. Data Augmentation

- Artificially enlarged the training dataset using:
 - **Image translation**
 - **Horizontal flipping**
 - **Patch extraction**
- Helped in reducing overfitting and improving generalization.

4. Use of GPUs

- AlexNet utilized **two Nvidia GPUs** for training.
- The model was split across GPUs to enable **data and model parallelism**.
- This significantly accelerated training time on large datasets like ImageNet.

5. Overlapping Pooling

- Used **stride = 2** with **window size = 3**.
- Unlike non-overlapping pooling (stride = 2, window size = 2), overlapping pooling retains more spatial information.
- Resulted in reduced error rates.

TensorFlow Optimizations

Prefetch in TensorFlow

- Prefetching overlaps **data preprocessing** and **model execution**.
- While the model is training on one batch, the **next batch is being prepared** in the background.
- This reduces model idle time and ensures **efficient use of CPU/GPU**.

✦ When to Use:

- Ideal when image loading, decoding, and transformations are **CPU-intensive**.
- Keeps the GPU busy and prevents bottlenecks due to slow input pipelines.

🤖 AUTOTUNE in TensorFlow

- `tf.data.AUTOTUNE` lets TensorFlow **dynamically tune** the `prefetch` buffer size.
- It decides how many batches to load in advance **based on available system resources**:
 - CPU memory
 - I/O bandwidth
 - Hardware configuration

✅ Benefits:

- **Reduced idle time** for training loops.
- **Automatic performance tuning** without manual buffer size tweaking.
- **Seamless and faster training**, especially for large datasets or heavy transformations.

These innovations not only made **AlexNet** a breakthrough in deep learning but also laid the groundwork for modern neural networks in terms of **performance, efficiency, and scalability**.

Setting Up Directories

```
In [3]: !mkdir -p ~/.kaggle/
        !cp kaggle.json ~/.kaggle/
```

```
In [4]: !kaggle datasets download -d salader/dogs-vs-cats
```

```
Warning: Your Kaggle API key is readable by other users on this system! To fix th
is, you can run 'chmod 600 /root/.kaggle/kaggle.json'
Dataset URL: https://www.kaggle.com/datasets/salader/dogs-vs-cats
License(s): unknown
Downloading dogs-vs-cats.zip to /content
 97% 1.03G/1.06G [00:07<00:00, 138MB/s]
100% 1.06G/1.06G [00:07<00:00, 146MB/s]
```

```
In [5]: !unzip /content/dogs-vs-cats.zip -d /content/
```

```

inflating: /content/train/dogs/dog.9976.jpg
inflating: /content/train/dogs/dog.9977.jpg
inflating: /content/train/dogs/dog.9978.jpg
inflating: /content/train/dogs/dog.9979.jpg
inflating: /content/train/dogs/dog.9980.jpg
inflating: /content/train/dogs/dog.9981.jpg
inflating: /content/train/dogs/dog.9982.jpg
inflating: /content/train/dogs/dog.9983.jpg
inflating: /content/train/dogs/dog.9984.jpg
inflating: /content/train/dogs/dog.9985.jpg
inflating: /content/train/dogs/dog.9987.jpg
inflating: /content/train/dogs/dog.9988.jpg
inflating: /content/train/dogs/dog.999.jpg
inflating: /content/train/dogs/dog.9990.jpg
inflating: /content/train/dogs/dog.9992.jpg
inflating: /content/train/dogs/dog.9993.jpg
inflating: /content/train/dogs/dog.9994.jpg
inflating: /content/train/dogs/dog.9996.jpg
inflating: /content/train/dogs/dog.9997.jpg
inflating: /content/train/dogs/dog.9998.jpg
inflating: /content/train/dogs/dog.9999.jpg

```

Installing the required libraries

In [28]: `# !pip3 install tensorflow numpy matplotlib keras`

Importing Packages

```

In [7]: import matplotlib.pyplot as plt
        from PIL import Image
        import random
        import os

        import tensorflow as tf
        from tensorflow.keras import layers, models
        import numpy as np
        import matplotlib.pyplot as plt
        from tensorflow.keras.preprocessing.image import ImageDataGenerator

        import random
        import matplotlib.pyplot as plt
        from tensorflow.keras.preprocessing.image import load_img
        from keras.utils import plot_model
        from sklearn.metrics import confusion_matrix, classification_report, roc_curve,
        import seaborn as sns
        from tensorflow.keras.preprocessing.image import img_to_array, load_img
        import numpy as np

        import tensorflow as tf
        from tensorflow.keras.preprocessing import image_dataset_from_directory
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
        from tensorflow.keras.optimizers import Adam

```

Setting up Path

```
In [8]: # Placeholder for the directory path
cat_directory_path = '/content/dogs_vs_cats/train/cats'
dog_directory_path = '/content/dogs_vs_cats/train/dogs'
```

Initialising the Generator

```
In [9]: # # Follow this if you have seperate folders -

# train_dir = ''
# validation_dir = ''
# image_size = (224, 224)
# batch_size = 100

# # Data augmentation for training set
# train_datagen = ImageDataGenerator(
#     rescale=1./255, # Normalize pixel values
#     rotation_range=20,
#     width_shift_range=0.2,
#     height_shift_range=0.2,
#     horizontal_flip=True,
#     fill_mode='nearest'
# )

# # Validation data without augmentation
# validation_datagen = ImageDataGenerator(rescale=1./255)

# # Generate data generators
# train_generator = train_datagen.flow_from_directory(
#     train_dir,
#     target_size=image_size,
#     batch_size=batch_size,
#     class_mode='binary'
# )

# validation_generator = validation_datagen.flow_from_directory(
#     validation_dir,
#     target_size=image_size,
#     batch_size=batch_size,
#     class_mode='binary'
# )
```

```
In [10]: # Set path to the dataset
data_dir = '/content/dogs_vs_cats/train'

# Hyperparameters
img_width, img_height = 224, 224 # AlexNet input size
batch_size = 32
epochs = 5
validation_split = 0.2 # 20% of data for validation
```

```
In [11]: # Load datasets
train_dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=validation_split,
    subset="training",
    seed=123,
    image_size=(img_width, img_height),
    batch_size=batch_size
)
```

Found 20000 files belonging to 2 classes.
Using 16000 files for training.

```
In [12]: validation_dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=validation_split,
    subset="validation",
    seed=123,
    image_size=(img_width, img_height),
    batch_size=batch_size
)
```

Found 20000 files belonging to 2 classes.
Using 4000 files for validation.

```
In [13]: # Data Augmentation Layer
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(0.2),
    tf.keras.layers.RandomZoom(0.2),
])
```

```
In [14]: # Apply data augmentation only on the training dataset
train_dataset = train_dataset.map(lambda x, y: (data_augmentation(x, training=True), y))
```

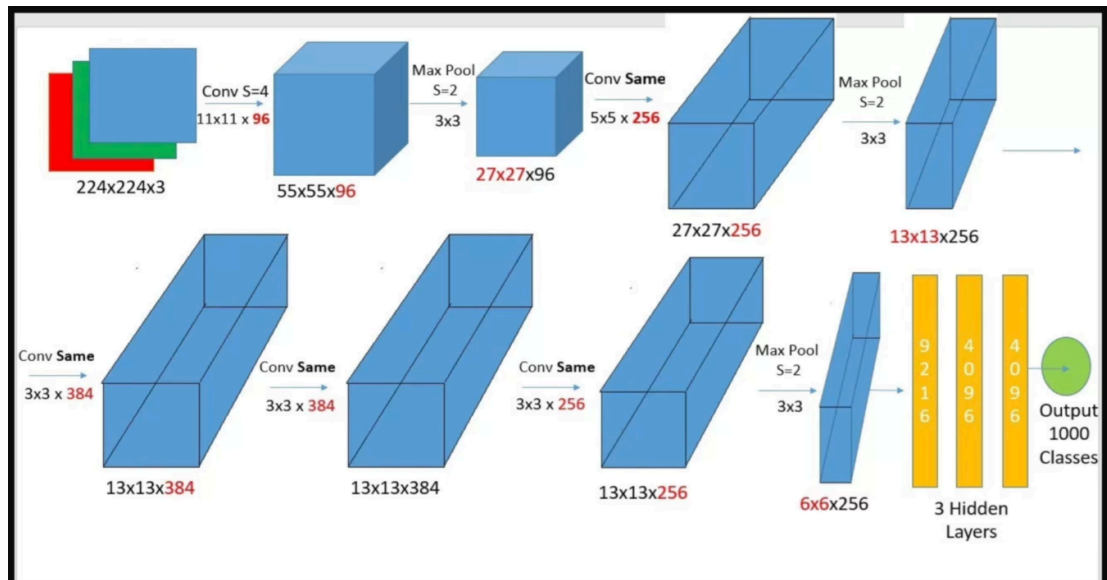
32 images - Pass to model training. load next batch of 32 images and - Pass to model training.

```
In [15]: # Prefetch the datasets for performance
AUTOTUNE = tf.data.AUTOTUNE # Performance of resources
train_dataset = train_dataset.prefetch(buffer_size=AUTOTUNE)
validation_dataset = validation_dataset.prefetch(buffer_size=AUTOTUNE)
```

Before diving into the specifics of AUTOTUNE, it's important to understand the role of data pipelines:

Data Loading: The process of reading data from storage (e.g., disk). Preprocessing: Operations like resizing, normalization, augmentation, etc., that are applied to the data before feeding it to the model. Batching: Grouping individual examples into batches for efficient processing. Prefetching: Loading the next batch of data while the current one is being processed by the model, to ensure that the GPU/CPU is always busy. A well-optimized data pipeline ensures that the model training process is not waiting for data, which would slow down the training.

Building Alexnet Model



```
In [16]: from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense,
# AlexNet model
alexnet_model = Sequential([
    Input(shape=(224, 224, 3)),

    Conv2D(96, (11, 11), strides=(4, 4), activation='relu'),
    MaxPooling2D((3, 3), strides=(2, 2)),

    Conv2D(256, (5, 5), padding='same', activation='relu'),
    MaxPooling2D((3, 3), strides=(2, 2)),

    Conv2D(384, (3, 3), padding='same', activation='relu'),
    Conv2D(384, (3, 3), padding='same', activation='relu'),
    Conv2D(256, (3, 3), padding='same', activation='relu'),

    MaxPooling2D((3, 3), strides=(2, 2)),

    Flatten(),

    Dense(4096, activation='relu'),
    Dropout(0.5),
    Dense(4096, activation='relu'),
    Dropout(0.5),

    Dense(1, activation='sigmoid')
])
```

```
In [17]: alexnet_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 54, 54, 96)	34,944
max_pooling2d (MaxPooling2D)	(None, 26, 26, 96)	0
conv2d_1 (Conv2D)	(None, 26, 26, 256)	614,656
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_2 (Conv2D)	(None, 12, 12, 384)	885,120
conv2d_3 (Conv2D)	(None, 12, 12, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 12, 12, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 256)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 4096)	26,218,496
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 1)	4,097



Total params: 46,751,105 (178.34 MB)

Trainable params: 46,751,105 (178.34 MB)

Non-trainable params: 0 (0.00 B)

Model Compile

```
In [18]: # Compile the model
alexnet_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['ac
```

Model Training

```
In [19]: history = alexnet_model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=1
)
```

500/500 ————— 212s 405ms/step - accuracy: 0.4939 - loss: 18.1946 - val_accuracy: 0.4915 - val_loss: 0.6938

```
In [20]: # Evaluate the model on validation data
val_loss, val_accuracy = alexnet_model.evaluate(validation_dataset)
```



```
print(f'Validation Accuracy: {val_accuracy * 100:.2f}%')
```

125/125  4s 33ms/step - accuracy: 0.4914 - loss: 0.6938
Validation Accuracy: 49.15%

```
In [21]: alexnet_model.save("saved_model_1_epochs.keras")
```

```
In [22]: alexnet_model.save_weights("model_1.weights.h5")
```

Model save_weights?

```
In [23]: from tensorflow.keras.models import load_model
```

```
# Load the model
```

```
loaded_model = load_model('saved_model_1_epochs.keras')
```

```
# Now you can use the loaded model for inference or further training
```

```
In [24]: loaded_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 54, 54, 96)	34,944
max_pooling2d (MaxPooling2D)	(None, 26, 26, 96)	0
conv2d_1 (Conv2D)	(None, 26, 26, 256)	614,656
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_2 (Conv2D)	(None, 12, 12, 384)	885,120
conv2d_3 (Conv2D)	(None, 12, 12, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 12, 12, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 256)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 4096)	26,218,496
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 1)	4,097



Total params: 140,253,317 (535.02 MB)

Trainable params: 46,751,105 (178.34 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 93,502,212 (356.68 MB)

```
In [25]: history = loaded_model.fit(
        train_dataset,
        validation_data=validation_dataset,
        epochs=2
    )
```

Epoch 1/2

500/500 ————— 203s 397ms/step - accuracy: 0.4905 - loss: 0.6937 -
val_accuracy: 0.4915 - val_loss: 0.6939

Epoch 2/2

500/500 ————— 193s 384ms/step - accuracy: 0.4985 - loss: 0.6935 -
val_accuracy: 0.4915 - val_loss: 0.7004

```
In [26]: history_dict=history.history
        history_dict
```

```
Out[26]: {'accuracy': [0.49524998664855957, 0.5027499794960022],
          'loss': [0.6935184597969055, 0.693476676940918],
          'val_accuracy': [0.49149999022483826, 0.49149999022483826],
          'val_loss': [0.69392991065979, 0.7004321813583374]}
```

```
In [27]: import matplotlib.pyplot as plt

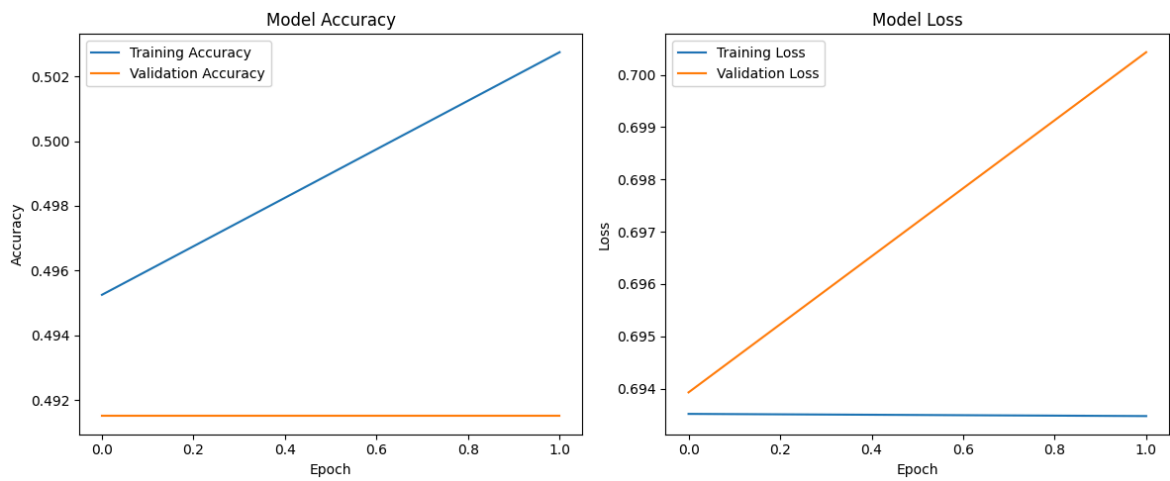
        # Assuming you have a `history` object returned from model.fit()
        history_dict = history.history # Access the history dictionary

        # Plotting accuracy
        plt.figure(figsize=(12, 5))

        plt.subplot(1, 2, 1)
        plt.plot(history_dict['accuracy'], label='Training Accuracy')
        plt.plot(history_dict['val_accuracy'], label='Validation Accuracy')
        plt.title('Model Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()

        # Plotting Loss
        plt.subplot(1, 2, 2)
        plt.plot(history_dict['loss'], label='Training Loss')
        plt.plot(history_dict['val_loss'], label='Validation Loss')
        plt.title('Model Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()

        # Show plots
        plt.tight_layout()
        plt.show()
```



```
In [ ]:
```